

Refatoração de Testes e Detecção de Test Smells

Capa

Disciplina: Teste de Software

Trabalho: Refatoração de Testes e Detecção de Test Smells

Aluno: Helio Ernesto Gouvea Dutra Teixeira

Repositório: <https://github.com/hegdt/test-smelly>

1. Análise de Smells Encontrados

A suíte original `test/userService.smelly.test.js` passa nos testes, mas possui problemas que reduzem sua clareza, dificultam manutenção e podem esconder falhas reais.

O primeiro smell identificado foi o **Eager Test** no teste `deve criar e buscar um usuário corretamente`. O mesmo teste valida criação e busca de usuário ao mesmo tempo. Quando um teste cobre mais de um comportamento, fica mais difícil entender qual requisito falhou caso o teste quebre. Na refatoração, esse cenário foi separado em dois testes: um para criação de usuário e outro para busca por `id`.

O segundo smell foi a presença de **lógica condicional no teste** no cenário `deve desativar usuários se eles não forem administradores`. O teste original usa `for` e `if` para validar usuários comuns e administradores dentro do mesmo fluxo. Isso deixa o teste mais complexo e faz com que algumas asserções só executem dependendo da condição. Esse problema também foi apontado automaticamente pelo ESLint com a regra `jest/no-conditional-expect`.

O terceiro smell foi o **teste frágil** no relatório de usuários. O teste original espera uma linha formatada com `ID`, nome, status e quebra de linha. Pequenas mudanças de formatação poderiam quebrar o teste mesmo que o comportamento principal continuasse correto. Na refatoração, o teste passou a verificar informações relevantes do relatório, como título, nomes de usuários e status, sem depender de toda a string exata.

Também foi encontrado um problema no teste de erro para usuário menor de idade. O teste original coloca o `expect` dentro do `catch`; se a exceção deixar de ser lançada, o teste pode passar silenciosamente. A versão limpa usa `toThrow`, garantindo que a ausência do erro faça o teste falhar.

2. Processo de Refatoração

Um dos testes mais problemáticos era o de desativação de usuários, pois misturava dois comportamentos e continha laço, condicional e expectativas condicionais.

Antes:

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois:

```
test('deve desativar um usuário comum', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

  const resultado = userService.deactivateUser(usuarioComum.id);
  const usuarioAtualizado = userService.getUserById(usuarioComum.id);

  expect(resultado).toBe(true);
  expect(usuarioAtualizado.status).toBe('inativo');
});

test('não deve desativar um usuário administrador', () => {
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  const resultado = userService.deactivateUser(usuarioAdmin.id);
  const usuarioAtualizado = userService.getUserById(usuarioAdmin.id);

  expect(resultado).toBe(false);
  expect(usuarioAtualizado.status).toBe('ativo');
});
```

A mudança separou as regras de negócio em testes independentes e mais legíveis. Cada teste segue o padrão Arrange, Act, Assert: primeiro prepara o usuário, depois executa `deactivateUser` e por fim verifica o resultado esperado. Com isso, não há laços, condicionais ou asserções que dependem de um caminho específico de execução.

3. Relatório da Ferramenta

Foi configurado o ESLint com `eslint-plugin-jest` no arquivo `.eslintrc.json`. Na primeira execução, usando `npx eslint .`, a ferramenta encontrou os seguintes problemas no arquivo original:

```
test/userService.smelly.test.js
44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
77:3  warning  Tests should not be skipped            jest/no-disabled-tests
77:3  warning  Test has no assertions                 jest/expect-expect

X 6 problems (4 errors, 2 warnings)
```

Esses avisos confirmaram parte da análise manual. A regra `jest/no-conditional-expect` detectou as expectativas dentro de fluxos condicionais, enquanto `jest/no-disabled-tests` e `jest/expect-expect` apontaram o teste pulado sem asserções.

Após a refatoração, o novo arquivo `test/userService.clean.test.js` foi validado com:

```
npx eslint test/userService.clean.test.js
```

O comando não retornou erros nem avisos para a suíte limpa.

4. Validação Final

A suíte completa foi executada com `npm test`. O resultado confirmou que os testes originais e os novos testes limpos passam:

```
PASS test/userService.clean.test.js
PASS test/userService.smelly.test.js

Test Suites: 2 passed, 2 total
Tests:       1 skipped, 12 passed, 13 total
```

O teste pulado permanece apenas no arquivo original, que foi preservado para comparação conforme solicitado. O arquivo limpo não contém testes pulados, expectativas condicionais ou cenários

excessivamente amplos.

5. Conclusão

A refatoração mostrou que uma suíte de testes pode passar e ainda assim apresentar baixa qualidade. Testes com smells tendem a ser mais difíceis de entender, manter e confiar. Ao aplicar o padrão Arrange, Act, Assert, separar cenários em testes menores e remover lógica condicional, os testes se tornam mais claros e comunicam melhor o comportamento esperado do sistema.

O uso do ESLint com o plugin do Jest complementou a análise manual, automatizando a detecção de problemas recorrentes. Essa combinação ajuda a manter a qualidade da suíte ao longo do tempo e reduz o risco de que testes frágeis escondam defeitos reais no código de produção.